

Maitrova AI Agent Full Flow and Server Guide

Code-derived description of how the website assistant and WhatsApp sales agent move data, call AI, search products, link accounts, update carts, and hand customers to secure checkout.

AUDIT BASIS

Repository: Full-Stack-Project

Reviewed: 15 September 2026

Scope: React + Node/Express + FastAPI + MongoDB + Gemini + Meta WhatsApp + Razorpay

Prepared from the current source tree. Secrets and private values are intentionally omitted.

Executive summary

The implemented solution is a multi-service commerce agent. The Python FastAPI service owns conversation reasoning, Gemini calls, WhatsApp webhook processing, durable jobs, handoff state, and product-search orchestration. The Node/Express service remains the authority for users, carts, orders, payments, product writes, inventory rules, and outbound order updates. MongoDB is the shared persistence layer. React is the customer storefront and completes authenticated account linking and checkout.

What must be running

Component	Role	Required?	Default / public access
React + Nginx	Storefront, login, cart, checkout, /whatsapp-connect	Yes for customer purchase flow	Dev 5173; prod HTTPS
Node/Express	Commerce authority, auth, cart, orders, Razorpay webhook, order notifications	Yes	5000; /api
FastAPI AI agent	Reasoning, WhatsApp webhook, queue worker, business/conversation APIs	Yes	8000; webhook /api/whatsapp/webhook
MongoDB	Commerce DB plus agent DB; durable queue and links	Yes	27017 or managed MongoDB
Gemini API	Intent parsing, response rewriting, image understanding	Recommended; rules fallback exists	Outbound HTTPS
Meta WhatsApp Cloud API	Inbound webhook and outbound messages/images	Yes for WhatsApp	Inbound HTTPS + outbound HTTPS
Razorpay	Payment and payment/refund webhook lifecycle	Yes for online payment	Outbound/inbound HTTPS

Current readiness finding

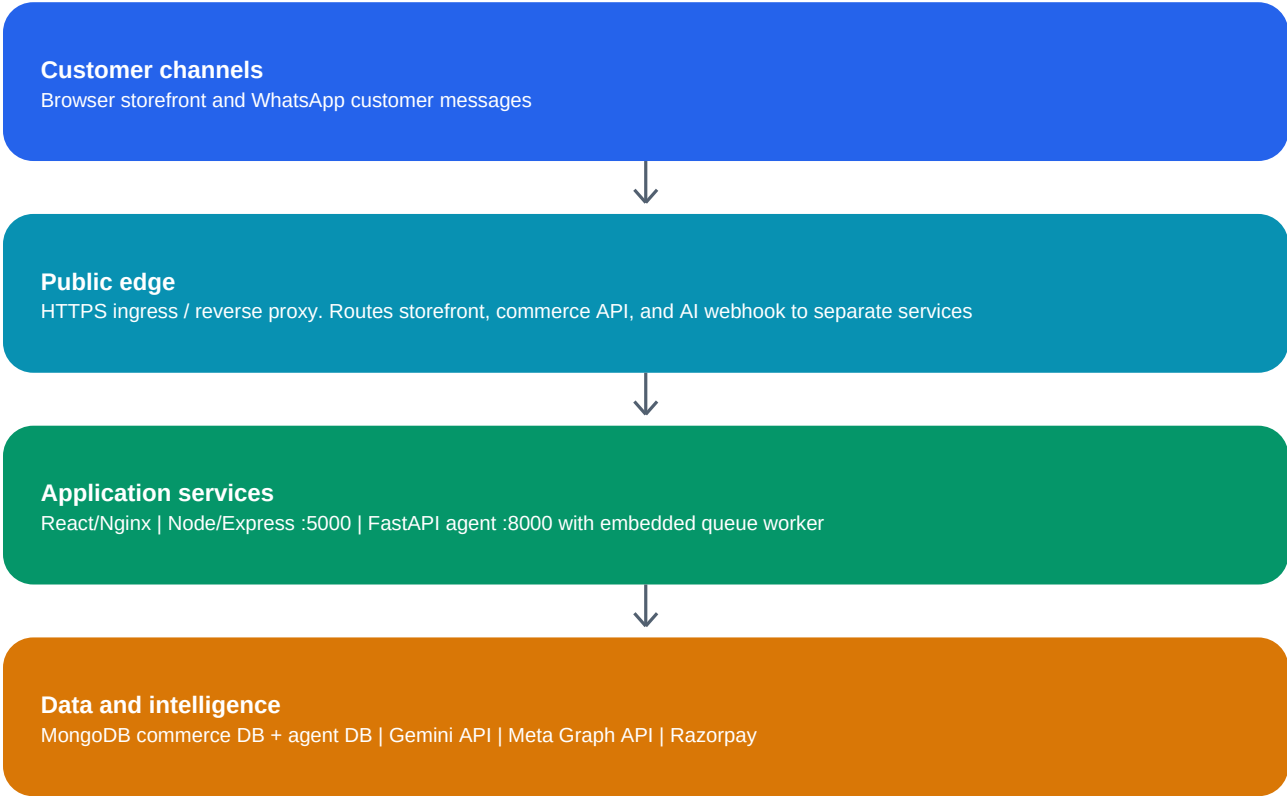
The WhatsApp path is substantially wired. The browser-chat path is not currently aligned: the React component calls /chat, /recommendations, and /chat/{session}, while FastAPI exposes /api/ai/chat and has no matching recommendation/history endpoints. The floating chat launch button is commented out. The Kubernetes charts do not include an AI-agent Deployment, Service, or Ingress. These are deployment blockers for the website assistant and production AI service.

Confidence statement

- Confirmed by code inspection: routing, database use, queue semantics, Gemini calls, account-link flow, cart idempotency, and deployment files.
- Verified locally: all 6 Node WhatsApp-commerce tests passed.
- Not executed: Python tests, because FastAPI and httpx are absent from the active Python environment. Install requirements first.
- Not proven by offline tests: real Meta delivery, public image reachability, real Gemini responses, live Razorpay callbacks, or deployed DNS/TLS.

1. System topology

Recommended production topology



Service boundaries

Owner	Reads / writes	Why the boundary matters
FastAPI	Agent businesses, conversations, messages, jobs, locks, delivery cache, handoff alerts; reads live catalogue	Conversation logic can evolve without becoming the system of record for payment or inventory.
Node/Express	Users, active carts, orders, inventory, coupons, payment state, link completion, notification log	All money and inventory mutations stay behind existing authenticated controllers and live validations.
React	Browser session/localStorage, signed-in user actions; calls APIs	No server secrets belong in Vite variables or browser code.
External providers	Gemini generation; Meta delivery; Razorpay payment events	Require outbound internet, credentials, quotas, and operational monitoring.

Database layout

FastAPI creates two Motor connections when URLs differ. MONGODB_URL/MONGODB_DB_NAME is the agent database (default ai_sales_employee). ECOMMERCE_MONGODB_URL/ECOMMERCE_MONGODB_DB_NAME is the live store database (default appdb). If both URLs are the same, one client is reused, but the logical database names can still differ. Node uses its own Mongoose connection to the commerce database.

2. Website assistant request flow



Actual implementation mismatch

React currently calls	FastAPI currently exposes	Result
POST /chat	POST /api/ai/chat	404 unless an external proxy rewrites both path and payload.
POST /recommendations	No matching route	Proactive recommendations fail silently.
GET/DELETE /chat/{sessionId}	No matching route	History load/reset fail silently.
Payload: session_id, message, page_context	AIChatRequest under authenticated /api/ai/chat	Schema/auth compatibility must be verified before exposing UI.

The component is mounted in App.jsx, but the button that opens it is commented out. Therefore the website assistant is effectively hidden even before route alignment is fixed.

Safe behavior when Gemini is unavailable

IntentParser falls back to rules for common category, color, occasion, fabric, price, Hindi, Telugu, and Roman-script phrases. ResponseGenerator returns its factual fallback draft. Image analysis returns no attributes. Text shopping can continue in a reduced mode, but image-based discovery and natural phrasing degrade.

3. WhatsApp inbound message flow



Important delivery semantics

- Duplicate inbound message IDs are ignored or replay their cached response.
- Per-sender locks keep messages from the same customer in order.
- Queue and progress survive service restarts because they live in MongoDB.
- A timeout after Meta accepted a send may still create a duplicate. This is at-least-once processing, not exactly-once delivery.
- When a conversation is in handoff status, later customer messages are stored without an automated reply.
- GET webhook verification uses WHATSAPP_VERIFY_TOKEN; POST signature verification uses the different WHATSAPP_APP_SECRET.

4. Product discovery and response logic

Decision pipeline

Stage	Behavior	Source of truth / fallback
Language	Detect Telugu script, Devanagari, Roman Telugu, Roman Hindi, or English	Local rules
Image	Gemini Vision extracts category/color/material/fabric/occasion/style/pattern/work/gender	Returns empty attributes if unavailable
Intent	Gemini returns constrained JSON: category, color, price, occasion, size, brand, attributes	Local rule parser on failure
Context	Merges current message with selected/recommended products and recent turns	Conversation document in agent DB
Product search	Loads active readymade catalogue, normalizes category/brand/images/variants, filters and sorts	Live ecommerce MongoDB
Actions	Details, stock, sizes, alternatives, comparisons, policy answer, order request, handoff	Tool results and deterministic code
Response	Gemini rewrites factual draft in the customer's language and WhatsApp style	Original draft if model fails

Guardrails in the prompt and code

- Only tool-returned products may be recommended; the model is told not to invent price, stock, size, fabric, URL, or availability.
- Option numbering and product order must remain stable so WhatsApp photos match the text.
- At most one useful question is requested; recent turns are context rather than new instructions.
- Policy answers use published HTML from company documents. Missing or PDF-only policy material is handed to staff instead of guessed.
- Custom products and combo packs are routed to storefront pages because their visual/configuration steps are not completed inside WhatsApp.

Catalogue performance note

EcommerceProductRepository currently loads up to 2,000 active products, joins category/subcategory/brand names, then filters in Python. This is simple and correct for a modest catalogue but becomes a scaling risk. Production growth should move normalized searchable fields and filters into indexed Mongo queries or a search service.

5. Secure WhatsApp purchase and checkout flow



Security properties

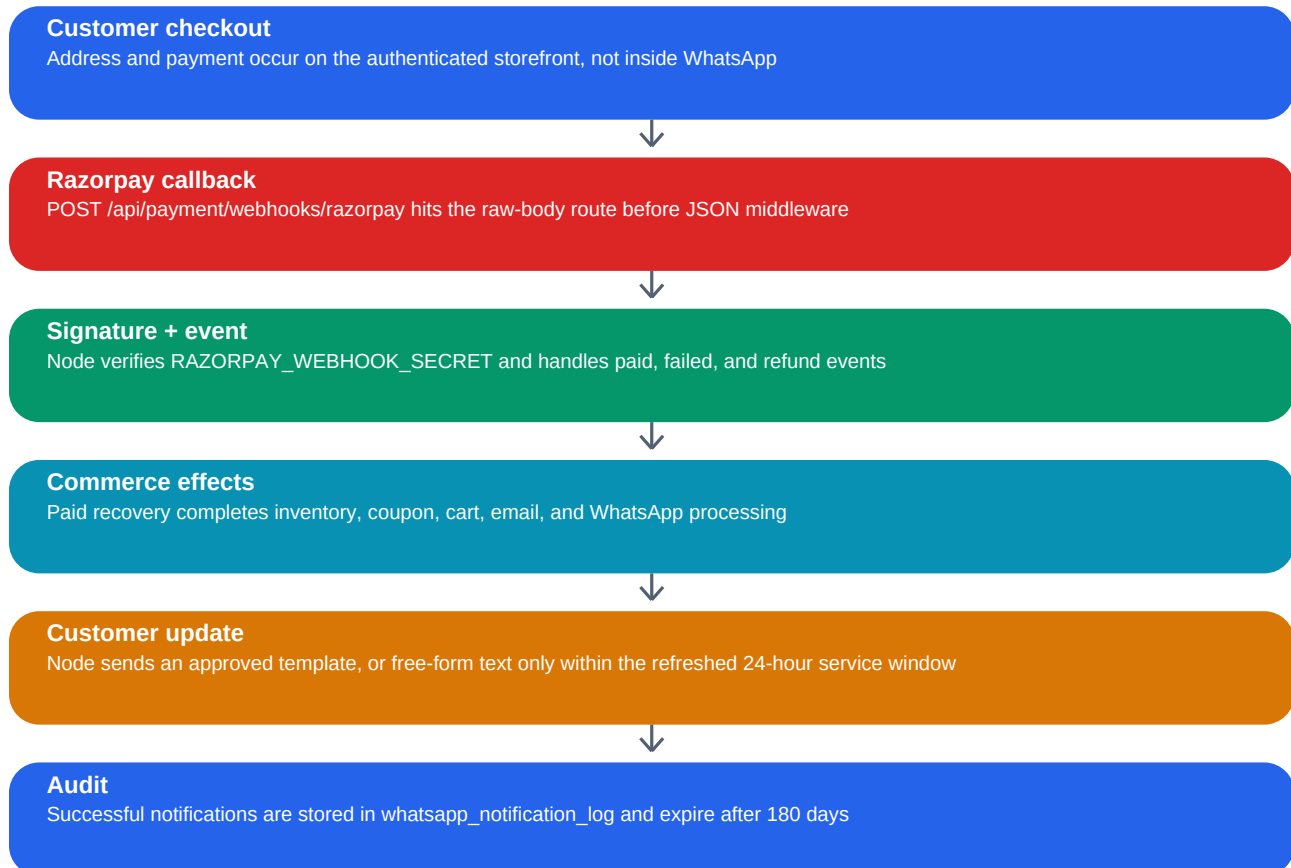
- The phone number is not accepted as account identity. The signed website session is the identity authority.
- The same WHATSAPP_COMMERCE_KEY must exist only on FastAPI and Node; Node compares it with timing-safe equality.
- A hashed account reference is required and must map to an unexpired link before orders or cart operations are allowed.
- Operation IDs and stored receipts prevent duplicate cart increments. Ambiguous pending operations return 409 for manual checking.
- Price and stock are checked again on Node immediately before cart mutation.
- STOP disables updates and deletes the link; disconnect removes the link; website revoke also disables subscriptions.
- Customers are explicitly told never to send OTPs, UPI PINs, or card details through WhatsApp.

6. Orders, payments, and notifications

Order lookup

The agent calls Node at `Ecommerce_API_URL + /whatsapp-commerce/orders` using `x-commerce-key` and `x-whatsapp-account`. Node resolves the linked user and returns only that user's five latest orders, or a specific 24-character order ID scoped to that user. The agent supports latest and second-most-recent wording.

Payment lifecycle



Meta messaging rule reflected in code

Every genuine inbound WhatsApp message refreshes the customer-service window for 24 hours. Outside that window, Node must use an approved transactional template. If no template is configured, an update is skipped with reason `template_required` rather than attempting a prohibited free-form message.

Notification ownership

FastAPI sends interactive agent replies and product media. Node sends asynchronous order/payment/refund status updates. Both use the same Meta phone number credentials, so credential rotation and Graph API versioning must be coordinated across both services.

7. API and persistence map

FastAPI public and staff endpoints

Endpoint	Purpose	Caller
GET /api/health	Process health	Load balancer / operator
GET /api/health/db	Database health	Readiness / operator
GET, POST /api/whatsapp/webhook	Meta verification and signed inbound events	Meta
POST /api/ai/chat	Authenticated agent chat	Intended website/admin client
/api/auth, /api/business, /api/products	Agent tenant and catalogue administration	Authenticated operator
/api/conversations/*	Conversation detail, human replies, handoff status	Staff UI / operator
GET /api/conversations/operations	Job and handoff counts	Monitoring
POST /api/conversations/operations/retry-failed	Requeue eligible failed jobs	Operator

Node WhatsApp-commerce endpoints

Endpoint	Protection	Purpose
POST /api/whatsapp-commerce/link/complete	User JWT + one-time token + rate limit	Connect account; optionally add confirmed item
DELETE /api/whatsapp-commerce/link	User JWT	Revoke link and updates
GET /api/whatsapp-commerce/orders	Commerce key + linked account	Latest orders
GET /api/whatsapp-commerce/orders/:id	Commerce key + linked account	One user-scoped order
POST /api/whatsapp-commerce/cart	Commerce key + linked account + idempotency	Validated cart mutation
POST /api/payment/webhooks/razorpay	Raw-body Razorpay signature	Payment/refund state transitions

Key MongoDB collections

Agent side: users, businesses, products (for native agent API), conversations, messages, orders, whatsapp_jobs, whatsapp_sender_locks, whatsapp_deliveries, whatsapp_outbound_context, whatsapp_rate_limits, whatsapp_handoff_alerts. Commerce side: readymadeproducts, users, carts, orders, companydocuments, whatsapp_link_requests, whatsapp_account_links, whatsapp_order_subscriptions, whatsapp_cart_operations, whatsapp_commerce_rate_limits, whatsapp_link_rate_limits, whatsapp_notification_log.

8. Configuration and secrets

Variable	Service	Purpose / production rule
MONGODB_URL / MONGODB_DB_NAME	FastAPI	Agent state. Use explicit production values.
ECOMMERCE_MONGODB_URL / _DB_NAME	FastAPI	Read live catalogue and shared WhatsApp commerce records.
MONGOOSE_URL	Node	Commerce database connection.
GEMINI_API_KEY / GEMINI_MODEL	FastAPI	Server-only Gemini access; model default is gemini-flash-lite-latest.
WHATSAPP_VERIFY_TOKEN	FastAPI	Meta GET webhook challenge token.
WHATSAPP_APP_SECRET	FastAPI	Meta POST signature verification; not the verify token.
WHATSAPP_ACCESS_TOKEN / PHONE_NUMBER_ID	Both	Send messages; rotate together and never expose to Vite.
WHATSAPP_GRAPH_API_VERSION	Both	Keep naming/version consistent (Python example uses v25.0).
WHATSAPP_BUSINESS_ID	FastAPI	Maps inbound channel to the configured business document.
WHATSAPP_COMMERCE_KEY	Both	Same random 32-byte secret for server-to-server authorization.
ECOMMERCE_API_URL	FastAPI	Reachable Node base ending in /api.
ECOMMERCE_STOREFRONT_URL	FastAPI	Public HTTPS origin used in secure customer links.
ECOMMERCE_PUBLIC_URL	FastAPI	Public origin serving stored product images.
JWT_SECRET	FastAPI	Agent API authentication; replace default.
FRONTEND_ORIGIN	FastAPI	Comma-separated CORS origins.
RAZORPAY_WEBHOOK_SECRET	Node	Dedicated Razorpay callback signature secret.
WHATSAPP_TEMPLATE_*	Node	Approved order/refund template names.
VITE_API_URL	React build	Public Node /api base; safe for browser.
VITE_AI_AGENT_API_URL	React build	Public FastAPI origin, after routes are aligned.

Secret-handling rules

- Store credentials in Kubernetes Secrets or an external secret manager, never in values.yaml, Git, logs, screenshots, or Vite variables.
- Use separate staging and production Meta, Gemini, Razorpay, JWT, MongoDB, and commerce-key credentials.
- Rotate any credential that has appeared in chat or source control.
- Restrict MongoDB networking so only Node, FastAPI, and authorized operators can connect.

9. How to run locally

Prerequisites

- Node.js matching the project dependencies, Python 3.13 recommended by the AI Dockerfile, and MongoDB.
- Two environment files: server/.env for Node and server/ai-agent-backend/.env for FastAPI.
- A browser-accessible Node API and FastAPI API. For real Meta webhooks, FastAPI needs a public HTTPS URL.

Start order

1. MongoDB

Start local MongoDB on port 27017, or use explicit managed MongoDB URLs.

2. Node commerce API

```
cd server
npm install
npm run dev
```

3. FastAPI Agent

```
cd server/ai-agent-backend
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

4. React storefront

```
cd frontend
npm install
npm run dev
```

5. Optional development tunnel

```
cloudflared tunnel --url http://localhost:8000
```

Configure the generated HTTPS URL in Meta as **https://.../api/whatsapp/webhook**. A transient tunnel is suitable only for development because its URL changes. Production needs stable DNS, TLS, and ingress.

Basic checks

```
FastAPI: GET http://127.0.0.1:8000/api/health
FastAPI DB: GET http://127.0.0.1:8000/api/health/db
Node: GET http://127.0.0.1:5000/health
Storefront: http://127.0.0.1:5173
```

10. Production deployment blueprint

Required workloads

Workload	Replicas	Probe	Notes
frontend	2+	HTTP /	Nginx static SPA; build with public API origins.
node-commerce	2+	GET /health	Shared MongoDB and shared persistent/object storage for /api/outputs.
ai-agent	Start with 1	GET /api/health and /api/health/db	Worker is embedded in every API process; concurrency behavior must be load-tested before scaling replicas.
mongodb	Managed replica set preferred	ping	Backups, encryption, authentication, monitoring.
ingress	HA	Provider specific	TLS; route three origins/paths; preserve raw webhook bodies.

Recommended routing

- www.maitrova.in -> frontend service.
- api.maitrova.in -> Node/Express service; /api/outputs remains publicly reachable for Meta image fetches.
- agent.maitrova.in -> FastAPI service; Meta webhook is /api/whatsapp/webhook.
- Set ECOMMERCE_STOREFRONT_URL=https://www.maitrova.in, ECOMMERCE_API_URL=https://api.maitrova.in/api, and ECOMMERCE_PUBLIC_URL=https://api.maitrova.in.

Deployment gap in this repository

The AI agent has a Dockerfile but no matching Helm Deployment, Service, Secret, HPA/PDB, or Ingress template. The frontend Dockerfile also does not declare VITE_AI_AGENT_API_URL as a build argument, so a production browser build cannot receive that value through the current Docker build contract. Add these before production rollout.

Scaling caution

The Mongo-backed job claim and sender lock are designed for multiple workers, but each Uvicorn process starts its own worker. Begin with one process/replica, test lease expiry, retries, duplicate delivery, and shutdown behavior, then scale deliberately. A later hardening step can split the worker into its own Deployment.

11. Observability and operations

Signal	Where	Alert / action
Failed WhatsApp jobs	whatsapp_jobs status=failed; operations endpoint	Alert on any sustained count; inspect error_type; retry only eligible jobs.
Old open handoffs	whatsapp_handoff_alerts	Alert on oldest unassigned age; route to store staff.
Webhook signature errors	FastAPI HTTP 403/503 logs	Check Meta app secret/config; never disable verification.
Meta delivery errors	FastAPI/Node logs	Inspect token, phone ID, Graph version, template status, and service window.
Razorpay webhook errors	Node logs and payment state	Verify raw-body route and secret; reconcile paid orders.
Queue age/attempts	whatsapp_jobs	Detect stuck worker, database latency, or provider outage.
Gemini fallback rate	FastAPI warning logs	Track model/API failures; text continues with rules but quality drops.
MongoDB health	/api/health/db and database metrics	Page on connection, latency, storage, replication, or backup failures.

Operator actions

- GET /api/conversations/handoffs to work open staff handoffs.
- GET /api/conversations/operations for job and handoff counts.
- POST /api/conversations/operations/retry-failed after the cause is corrected.
- Change a conversation to OPEN to resume automated replies; a human reply assigns the alert.
- For an ambiguous cart operation, ask the customer to inspect the cart before issuing a new operation.

Failure-mode behavior

Failure	Customer/system behavior
Gemini unavailable	Rule parsing and factual fallback text continue; image analysis is skipped.
Node unavailable	Product reads may still work through MongoDB, but linked cart/order actions return a temporary service message.
MongoDB unavailable at startup	FastAPI startup fails, preventing unsafe stateless processing.
Meta send fails	Job retries up to five attempts; duplicates remain possible after uncertain timeout.
No staff policy content	Agent offers human handoff instead of inventing policy.
Expired account link	Agent issues a fresh token after revalidating product stock and price.

12. Release checklist and verified findings

Blockers to fix before calling the full agent live

- Align browser endpoints and schemas: either change React to the real FastAPI paths/contracts or add compatible FastAPI routes.
- Implement /recommendations and chat history endpoints, or remove those calls and document the reduced feature set.
- Restore the AI chat launcher button if website chat is intended to be customer-visible.
- Add VITE_AI_AGENT_API_URL to the frontend Docker build arguments and deployment pipeline.
- Add a dedicated AI-agent Kubernetes Deployment, Service, Secret wiring, probes, and ingress.
- Install Python requirements and run the four documented Python test modules successfully.
- Test the complete flow on a Meta test number and staging store before customer traffic.

Acceptance tests

Area	Must prove
Webhook security	Valid challenge; reject missing/invalid signatures; reject oversized payload.
Conversation	Text, image, language matching, follow-up selection, duplicate inbound, ordered sender processing.
Commerce	Link, expiry, revoke, STOP, cross-account denial, stock change, price change, duplicate operation.
Checkout	Login return, cart add exactly once, address, Razorpay, eligible COD, order creation.
Lifecycle	Payment failure/recovery, shipped/delivered/cancel/refund templates and service-window behavior.
Operations	Worker restart, retry cap, handoff pause/resume, logs, alerts, backups.
Public assets	Meta can fetch every product image URL over HTTPS without authentication.

Test result from this audit

PASS: Node command `node --experimental-vm-modules --test tests/whatsappCommerce.test.mjs` completed 6/6 tests.
NOT RUN: Python tests failed during import because fastapi and httpx are not installed in the active Python environment. This indicates missing local dependencies, not a failing test assertion.

Source files used for this report

server/ai-agent-backend/app/main.py; config/settings.py; database/mongodb.py; routes/ai.py; routes/whatsapp.py; ai/sales_agent.py; ai/intent_parser.py; ai/response_generator.py; ai/gemini_client.py; ai/product_image_analyzer.py; services/whatsapp_queue.py; services/whatsapp_service.py; services/whatsapp_commerce.py; repositories/ecommerce_product_repository.py; tools/product_tools.py; OPERATIONS.md; .env.example; Dockerfile; server/server.js; server/routes/whatsappCommerce.js; server/services/whatsappOrderService.js; frontend/src/components/AIShoppingChat.jsx; frontend/src/pages/WhatsAppConnect.jsx; Helm charts and Dockerfiles.

Bottom line

Run MongoDB, Node/Express, FastAPI, and the React storefront as separate services. Expose only HTTPS public endpoints, keep secrets server-side, and let Node remain the authority for authenticated commerce. The WhatsApp flow can be staged after credentials and ingress are configured. The website AI chat requires route/contract and deployment fixes before it will operate end to end.